

*I Wanna Be
Like You*

Jack
Fraser-Govil

Why Emulate?

Standard
Estimators

Standard
Predictors

Models into
Emulators

The FADE
Emulator

I Wanna Be Like You

Robust & Rapid Emulation for Efficient Trial Design

Jack Fraser-Govil

23/03/26

*I Wanna Be
Like You*

Jack
Fraser-Govil

Why do we want to Emulate?

Why Emulate?

Standard
Estimators

Standard
Predictors

Models into
Emulators

The FADE
Emulator



UNIVERSITY OF
BIRMINGHAM

Why Emulate?

Standard
Estimators

Standard
Predictors

Models into
Emulators

The FADE
Emulator

- Suppose we have some computational model of the effect of an intervention

$$\mathbf{y} = \hat{M}(\boldsymbol{\pi})$$

Why Emulate?

Standard
Estimators

Standard
Predictors

Models into
Emulators

The FADE
Emulator

- Suppose we have some computational model of the effect of an intervention

$$\mathbf{y} = \hat{M}(\boldsymbol{\pi})$$

- Then suppose we wish to compute some complex quantity - such as the predictive power

$$p = \sum_{\text{permutations}} \sum_{\boldsymbol{\pi}} f(\mathbf{y}(\boldsymbol{\pi}))$$

Why Emulate?

Standard
Estimators

Standard
Predictors

Models into
Emulators

The FADE
Emulator

- Suppose we have some computational model of the effect of an intervention

$$\mathbf{y} = \hat{M}(\boldsymbol{\pi})$$

- Then suppose we wish to compute some complex quantity - such as the predictive power

$$p = \sum_{\text{permutations}} \sum_{\boldsymbol{\pi}} f(\mathbf{y}(\boldsymbol{\pi}))$$

The problem

If $\hat{M}$ is slow (minutes-to-hours), it will take years-to-decades to compute p .

Why Emulate?

Standard
Estimators

Standard
Predictors

Models into
Emulators

The FADE
Emulator

What is an Emulator?

An emulator is a function $\hat{E}_M$ which *behaves* like $\hat{M}$, but which is vastly quicker to compute.

Why Emulate?

Standard
Estimators

Standard
Predictors

Models into
Emulators

The FADE
Emulator

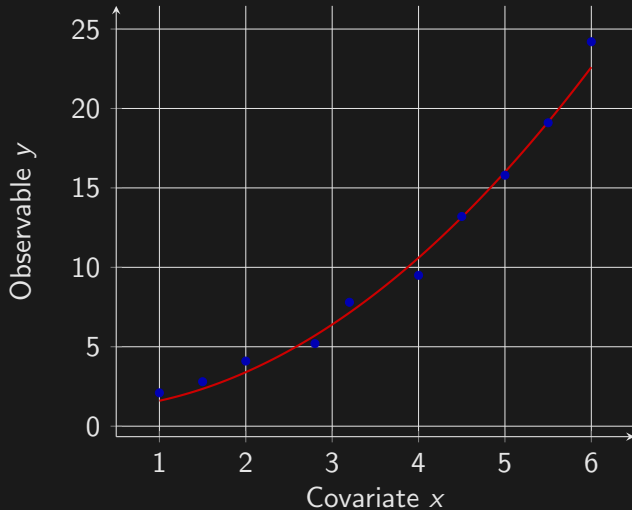
What is an Emulator?

An emulator is a function $\hat{E}_M$ which *behaves* like $\hat{M}$, but which is vastly quicker to compute.



I wanna walk like you, talk like you...

This is not a new idea!



In recent years¹, we can use machine learning

- Vast non-parametric models

¹Though even this is becoming 'old'

In recent years¹, we can use machine learning

- Vast non-parametric models
- Condition it on observable data



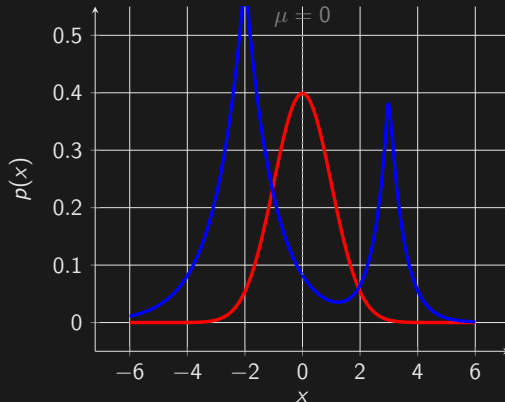
*Now give me the secret, mancub
C'mon, clue me what to do*

- Hope it learns the correct behaviour

¹Though even this is becoming 'old'

Distributions, not means

If $\hat{M}$ is stochastic, then $\mathbf{y}$ is not a number - it is a *realisation on a random process*. These methods serve only to predict the mean, $\mathbb{E}(\mathbf{y})$, not the distributions around that point.



There *are* ML models which give distributions (Bayesian Neural Networks), but we may choose to avoid them:

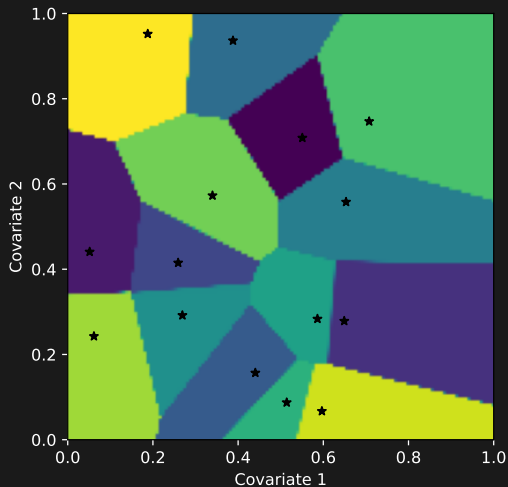
- ① Require lots of data (expensive - recall computing $\hat{M}$ is the bottleneck)
- ② Black boxes (aesthetically/scientifically dubious)

A modelling process where one assumes

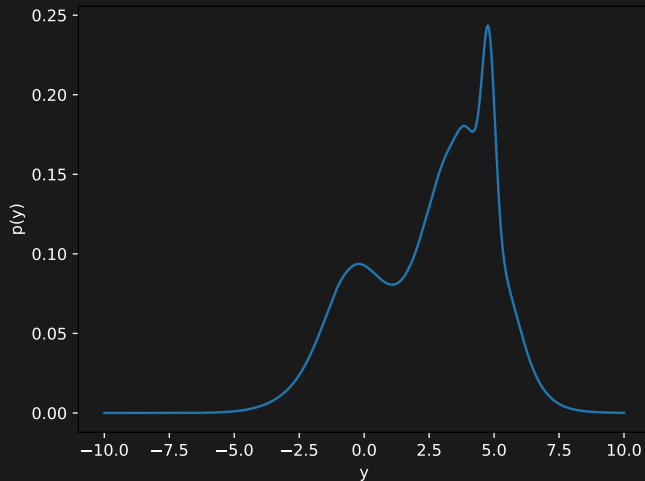
$$\mathbf{y}(\boldsymbol{\pi}) \sim \mathcal{N} \left(\sum_i w_i(\boldsymbol{\pi}) \mathbf{y}_i, \boldsymbol{\Sigma} \right) \quad (1)$$

Has many lovely properties, but fundamentally unsuitable for our work, as it assumes the distribution, rather than reconstructing it!

Split the covariate space up into *cells*



Each cell fits their own PDF:



Instead of a hard partition, MoE probabilistically assign the ‘problem’ to an ‘expert’:

$$p(y|\mathbf{x}) = \sum_i w_i(\mathbf{x}) p_i \quad (2)$$

The w_i give different weights to the ‘expert opinion’ at different points in covariate space, coming to a different consensus.

Standard MoE uses Neural Networks to determine w_i (via complex partitioning of a learned hyperplane projection)

Important!

The above models *are not emulators*. They provide ways to model the best-fit posterior distribution, conditioned on the observed data:

$$\text{Model}_{\Theta}(\mathbf{x}) = p(\mathbf{y}|\mathbf{x}, \Theta)$$



*What I desire is man's red fire
To make my dream come true*

What we need is the *posterior predictive* distribution. The distribution conditioned *only* on the data:

What we need is the *posterior predictive* distribution. The distribution conditioned *only* on the data:

$$\text{Emulator}(\mathbf{x}) = p(\mathbf{y}|\mathbf{x}) = \int \text{Model}_{\Theta}(\mathbf{x}) d\Theta \quad (3)$$

What we need is the *posterior predictive* distribution. The distribution conditioned *only* on the data:

$$\text{Emulator}(\mathbf{x}) = p(\mathbf{y}|\mathbf{x}) = \int \text{Model}_{\Theta}(\mathbf{x}) d\Theta \quad (3)$$



*An ape like me
Can learn to be human too*

*I Wanna Be
Like You*

Jack
Fraser-Govil

Why Emulate?

Standard
Estimators

Standard
Predictors

Models into
Emulators

The FADE
Emulator

We needed an emulator which can:

We needed an emulator which can:

- 1 Reconstruct probability distribution function

We needed an emulator which can:

- ① Reconstruct probability distribution function
- ② Allow both smooth and near-discontinuous changes in behaviour

We needed an emulator which can:

- ① Reconstruct probability distribution function
- ② Allow both smooth and near-discontinuous changes in behaviour
- ③ Inherit 'expert knowledge' to reduce training requirements

We needed an emulator which can:

- ① Reconstruct probability distribution function
- ② Allow both smooth and near-discontinuous changes in behaviour
- ③ Inherit 'expert knowledge' to reduce training requirements
- ④ Be interpretable and accessible

A middle ground between partitioning and MoE models. We allow each expert to belong to a 'department' (a partition - which may have many experts within it), but also allow them to 'spend time' between departments.

The weight of an expert in the sum is then:

$$w_i(\mathbf{x}) = \sum_{\text{dep. } k} P(\mathbf{x} \in k) \times \frac{P(i \in k)P(i \text{ knows } \mathbf{x})}{\sum_j P(j \in k)P(j \text{ knows } \mathbf{x})} \quad (4)$$

These probabilities can be learned by the machine

This reduces trivially down into a standard MoE, with model parameters Θ

$$p(\mathbf{x}|\Theta) = \sum_w w_i(\mathbf{x}|\Theta) p_i(\mathbf{x}|\Theta) \quad (5)$$

The clever bit is that we allow each department to have their own ‘grammar’ that they use to talk to interpret the covariance space. The distance metric in each space is:

$$d_k(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y}) \cdot G_k(\mathbf{x} - \mathbf{y})} \quad (6)$$

Where G_k is a learned distance metric unique to each cell.

Why is this clever?

*I Wanna Be
Like You*

Jack
Fraser-Govil

Why Emulate?

Standard
Estimators

Standard
Predictors

Models into
Emulators

The FADE
Emulator



- 1 Allows w_i to encode highly nonlinear relationships in $\mathbf{x}$ *without resorting to black-box machinery.*

- 1 Allows w_i to encode highly nonlinear relationships in $\mathbf{x}$ *without resorting to black-box machinery.*
- 2 The metrics are easily interpreted as covariance measures localised within a department.

- ① Allows w_i to encode highly nonlinear relationships in $\mathbf{x}$ *without resorting to black-box machinery.*
- ② The metrics are easily interpreted as covariance measures localised within a department.
- ③ Gives the same predictive power as a high dimensional hidden-layer neural network

- 1 Allows w_i to encode highly nonlinear relationships in $\mathbf{x}$ *without resorting to black-box machinery*.
- 2 The metrics are easily interpreted as covariance measures localised within a department.
- 3 Gives the same predictive power as a high dimensional hidden-layer neural network
- 4 Constrained to produce 'interpretable results'.

Why Emulate?

Standard
Estimators

Standard
Predictors

Models into
Emulators

The FADE
Emulator

